

Evulator

Genomic Computation, DNA-Native AI,
and Adaptive Machine Intelligence

Canonical deep edition

Chapter 1: Why Genomic Computation?

Hermon AI / coordinated research-textbook series

Internally reviewed development prototype / 7 September 2026

About this prototype

This is the first chapter of the canonical deep edition, not a completed book. The earlier introductory experiment is frozen; it may later inform a derivative, but it is not maintained chapter by chapter.

The chapter assumes technical maturity in programming and elementary mathematics. Domain-specific ideas are explained through mechanisms, original diagrams, formal definitions and worked examples. Useful abstraction is part of the explanation, not something to avoid.

All figures are original editable vectors with semantic TXT companions. Static keyframes are included in the repository; they are not exported animations. Exact numerical examples come from tested Python. No new laboratory experiment, trained DNA-native model or production inference engine is reported.

Review is internal author-agent scientific, mathematical, clarity, citation and rendered-page review. Independent specialist review and observation of real readers remain open. This PDF is not tagged and is not claimed to meet PDF/UA.

Contents

1	Why Genomic Computation?	1
1.1	Three ways to organize computation	1
1.2	Stored, expressed, and adapted are different roles	2
1.3	What biological regulation actually contributes	3
1.4	An exact typed example of expression	4
1.5	Development: constructing the working system	6
1.6	Execution, learning, and structural change	7
1.7	Before calling it genomic, check computer science	8
1.8	Evolutor, DOGMA, and Hermon DNA	9
1.9	Two memory strategies, one honest budget	11
1.10	A research program with concrete stopping points	12
1.11	Exercises	13
1.12	Worked solutions and discussion	14
	Chapter 1 glossary	15
	Bibliography	16
	Index	17

CHAPTER 1

Why Genomic Computation?

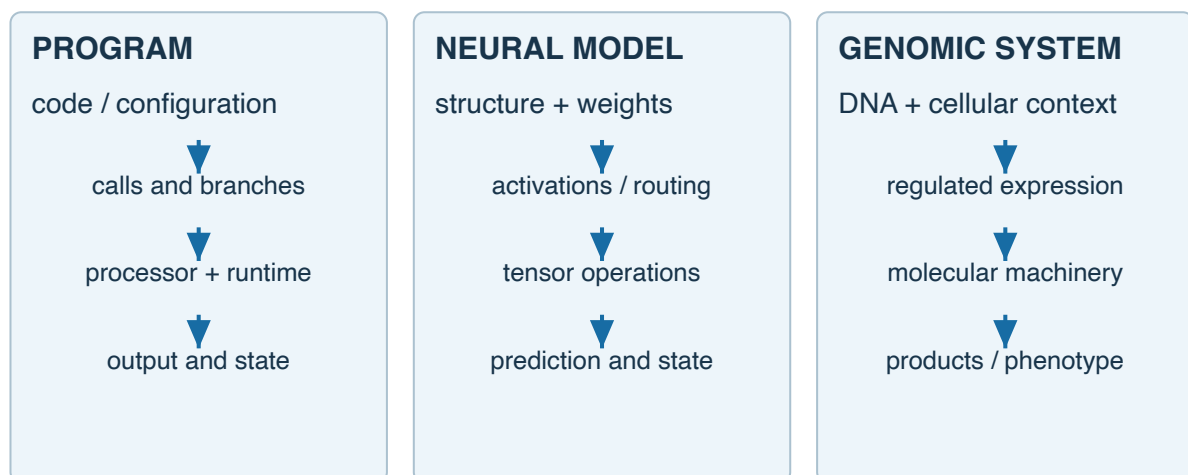
The research question

A genome can persist while different parts participate in different activities. Can an engineered separation between stored structure, expressed computation, and structural adaptation yield a useful computational organization beyond ordinary routing and program construction? This chapter makes the question precise without assuming the answer.

1.1 Three ways to organize computation

Imagine a system asked to behave differently in two environments while retaining a common internal library. A program can branch. A neural model can change its activations or route work. A cell can alter gene expression. Each description involves persistent structure and context-dependent activity, but their mechanisms are not interchangeable.

The comparison becomes useful when we ask the same operational questions: what is stored, what executes, what selects the activity, what remains afterward, and what can change the stored organization? Figure 1.1 provides an entry map. The table below sharpens it without treating one column as a replacement for the others.



Aligned questions do not make the three mechanisms identical.

Figure 1.1: Three organizations compared through persistence, control, activity, and change. A program, a neural model, and a biological genomic system can all separate stored information from current activity. Similar roles do not imply identical mechanisms or equivalent capabilities.

Question	Program	Neural model	Biological genomic system
What persists?	Source, binary, configuration, saved data	Architecture, weights, optionally saved state	Genome; inherited and cellular context also matter
What executes?	Instructions and invoked routines	Numerical operations producing activations	Molecular processes involving RNA, proteins, and other components
What selects?	Branches, dispatch, scheduler, inputs	Activations, attention, gates, routing	Regulatory interactions in a cellular environment
What adapts?	Whatever update procedures are implemented	Parameters, state, or structure under specified update rules	Physiological and regulatory changes; population evolution across generations
What changes structure?	Code/configuration changes or generated programs	Architecture search, pruning, growth, redesign	Variation, inheritance, selection, and developmental processes

The biology column is an orientation, not a universal inventory of every organism. Gene expression can yield functional RNA or protein; control occurs at multiple stages, not only through an on/off switch at a DNA site.[1, 2] The software columns are likewise broad: conventional programs can be stateful, reflective, adaptive, or generated by other programs.

This immediately rules out a weak argument for Evolutor. Merely observing that stored components outnumber active components does not establish a new kind of computation. The research must identify a mechanism, a useful property, and a comparison under which that property matters.

1.2 Stored, expressed, and adapted are different roles

Let G denote a persistent computational description, c a context, x an input, and s a runtime state. A proposed abstraction separates selection, expression, and execution:

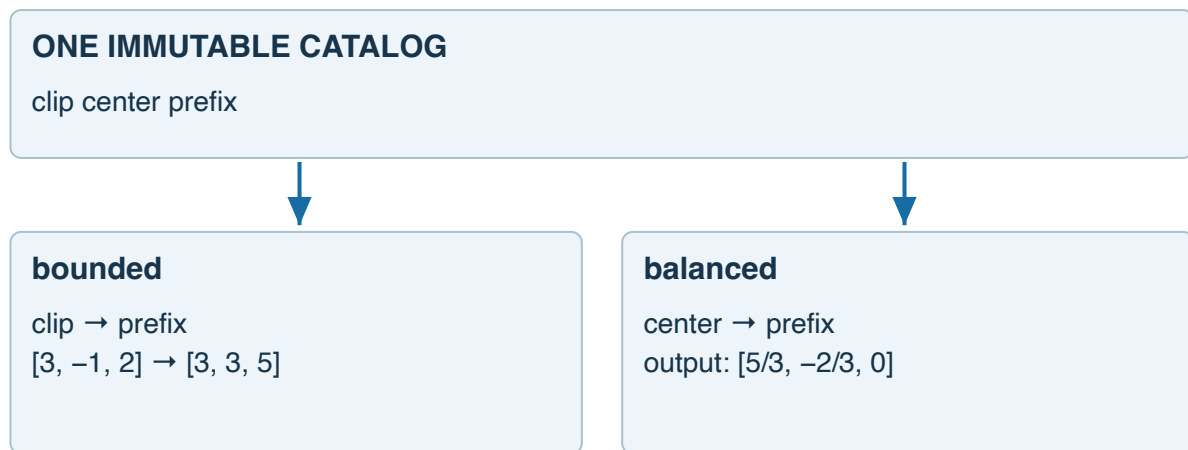
$$\begin{aligned}
 r &= \text{Reg}(G, c, s), \\
 P &= \text{Expr}(G, r), \\
 (y, s') &= \text{Exec}(P, x, s).
 \end{aligned}$$

Here r is a control decision, P is an executable plan, y an output, and s' the next state. These are candidate software interfaces, not equations of cell biology.

Stored structure describes what the system could use. **Expressed computation** is the selected or constructed activity for this occasion. **Adapted structure** is a later persistent description G' produced by an update process. The shorthand

$$\text{stored structure} \neq \text{expressed computation} \neq \text{adapted structure}$$

distinguishes roles and times, not values that must always be numerically unequal. An update may leave $G' = G$; an expression rule may activate the entire catalog.



Same catalog and input; context changes the selected plan.

Figure 1.2: One immutable catalog supports two plans for the same input. Only the context changes. This exact routed-program example illustrates stored versus active structure; it does not implement learning, biological expression, or a new architecture.

Separating these roles may help with inspection and controlled change. We can inspect a plan before execution, log which modules ran, and restrict what an update may alter. But those are also familiar benefits of interpreters, workflows, and modular software. The notation creates a place to ask questions; it does not answer the novelty question.

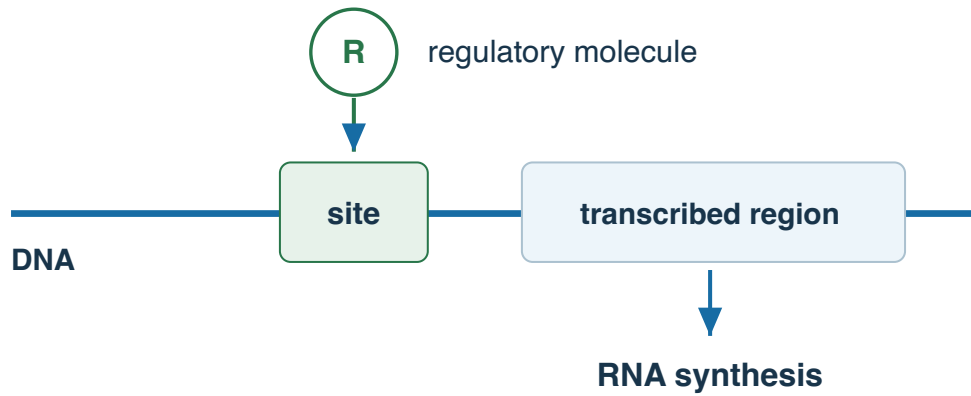
The crucial distinction is counterfactual. If we change the context while holding G fixed, we test regulation. If we change G and replay the same context and input, we test persistent structural change. If neither intervention is possible or observable, describing the system as adaptive may hide rather than clarify its behavior.

1.3 What biological regulation actually contributes

A genome is not a list of Python functions. Its sequence participates in a cell that already contains molecular machinery, spatial organization, and a history. Many differentiated cells can carry substantially the same genome while exhibiting different expression patterns; exceptions such as immune-cell DNA rearrangements prevent an absolute equivalence claim.[2, 3]

Gene expression produces functional products from genomic information. Transcription produces RNA; some RNAs function directly and others serve as templates for protein production.[1] Regulation can affect transcription, RNA processing or stability, translation, and protein activity.[2] Thus a useful computational analogy must not reduce expression to fetching one stored object unchanged.

A qualitative transcription-control example



Binding can change activity; the DNA sequence need not change.

Other control points exist; no quantitative rates are shown.

Figure 1.3: A qualitative transcription-control sketch. A regulatory molecule interacts with a DNA-associated site and can alter activity without rewriting the DNA sequence. The figure omits detailed complexes, kinetics, and other control points; the arrow means an interaction, not a software call.

The computational question extracted from this biology is narrower than a claim of imitation: can a persistent description support context-sensitive production of working structure with tractable rules for reuse, interference, and change? In software, we are free to design that interface differently from a cell. What we may not do is use the biological origin of an analogy as evidence of engineering performance.

A switch choosing one of two branches is a valid model of one control decision. It is not a quantitative gene-regulatory network. To justify that stronger description, we would need states, interactions, rates or transition rules, an account of feedback, and evidence that the abstraction preserves the biological property of interest. Chapter 1 introduces the distinction; later biological chapters develop the mechanisms.

1.4 An exact typed example of expression

We now build a deliberately ordinary program. Its simplicity is useful: every change is visible, and it has an exact conventional baseline.

Let

$$\mathcal{X} = \bigcup_{m \geq 1} \mathbb{Q}^m$$

be the type of nonempty finite rational sequences. For $x = (x_1, \dots, x_m)$ define three length-preserving modules:

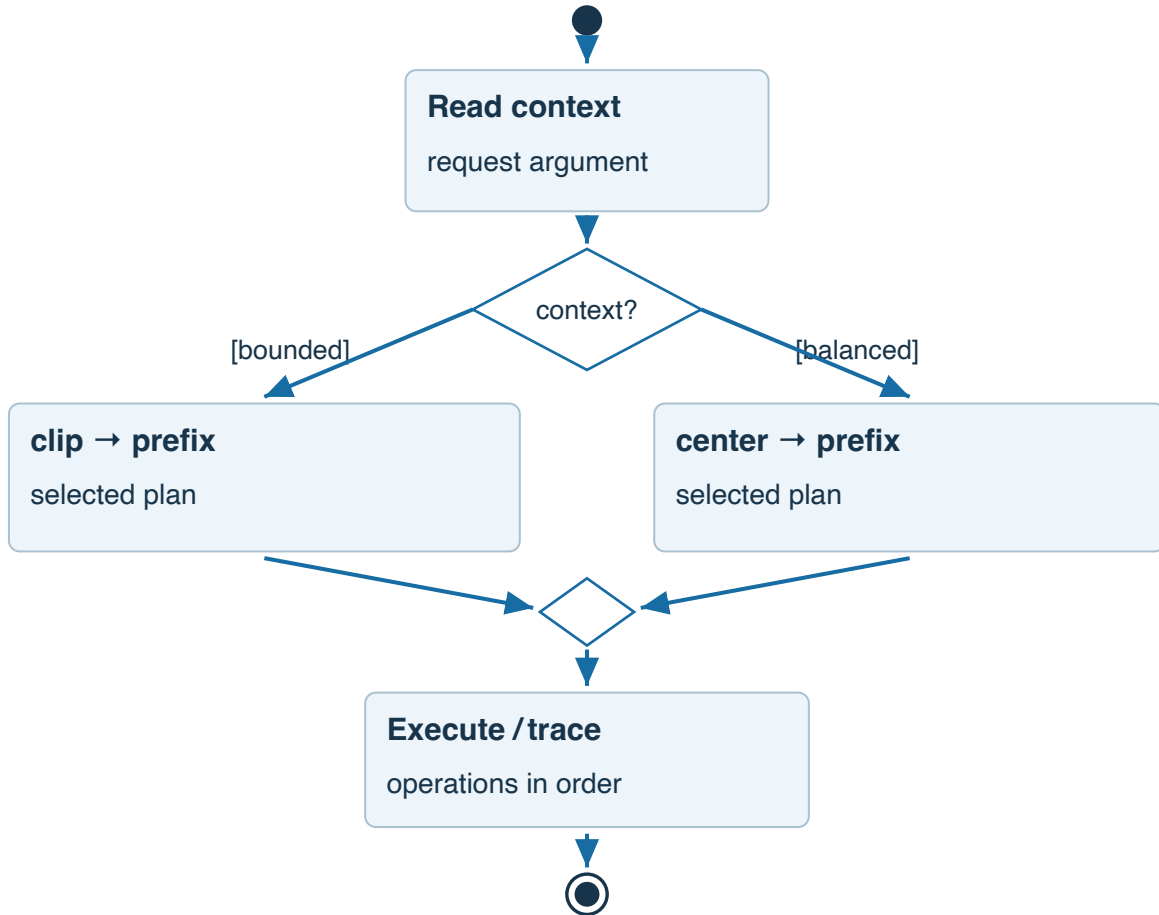
$$\begin{aligned} \text{clip}(x)_i &= \max(0, x_i), \\ \text{center}(x)_i &= x_i - \bar{x}, & \bar{x} &= \frac{1}{m} \sum_{j=1}^m x_j, \\ \text{prefix}(x)_i &= \sum_{j=1}^i x_j. \end{aligned}$$

Each module has type $\mathcal{X} \rightarrow \mathcal{X}$. A plan is a finite ordered list of module names. Its meaning is their left-to-right composition. The shared type makes every such composition well-typed; it does not make every composition useful.

The context set is $\mathcal{C} = \{\text{bounded}, \text{balanced}\}$. Our selection rule is

$$\text{Reg}(c) = \begin{cases} (\text{clip}, \text{prefix}), & c = \text{bounded}, \\ (\text{center}, \text{prefix}), & c = \text{balanced}. \end{cases}$$

The catalog is immutable. There are no learned parameters. Context is an explicit argument, not inferred from hidden state.



UML activity; unknown context raises an error (not shown).

Figure 1.4: UML activity-style control flow for the worked example. Guarded alternatives select one ordered plan; execution records intermediate values. Unknown contexts raise an error outside the two successful branches shown. This is software control flow, not a biochemical pathway.

For $x = (3, -1, 2)$, the bounded route gives

$$(3, -1, 2) \xrightarrow{\text{clip}} (3, 0, 2) \xrightarrow{\text{prefix}} (3, 3, 5).$$

For the balanced route, $\bar{x} = 4/3$, so

$$(3, -1, 2) \xrightarrow{\text{center}} (5/3, -7/3, 2/3) \xrightarrow{\text{prefix}} (5/3, -2/3, 0).$$

Exact rational arithmetic makes these results transparent. The generated table comes directly from the executable source:

Context	Plan	Output
bounded	clip / prefix	3, 3, 5
balanced	center / prefix	5/3, -2/3, 0

The two contexts express different mathematical invariants. The bounded output is nonnegative and nondecreasing because it sums nonnegative terms. The balanced output ends at zero because the centered values sum to zero. Neither invariant says the intermediate balanced prefix must be nonnegative. The negative second entry is correct.

Order matters. Applying prefix before clip to $(3, -1, 2)$ yields $(3, 2, 4)$, not $(3, 3, 5)$. A set of active names cannot represent this distinction; an ordered plan can. This is the first reason to specify expression semantics more precisely than an active/inactive mask.

The source below records both input and output of each operation. The excerpt is generated from the tested file, `examples/deep/ch01.py`, not independently retyped.

```

20 def select(context):
21     if context=="bounded":
22         return ("clip","prefix")
23     if context=="balanced":
24         return ("center","prefix")
25     raise ValueError("unknown context")

27 def execute(plan, values):
28     xs=tuple(Fraction(x) for x in values)
29     if not xs:
30         raise ValueError("nonempty rational sequence required")
31     trace=[]
32     for name in plan:
33         if name not in CATALOG:
34             raise ValueError("unknown module")
35         before=xs
36         xs=CATALOG[name](xs)
37         trace.append({"module":name,"before":before,"after":xs})
38     return xs,trace

```

A direct branching implementation computes the same specification without a plan interpreter. Tests compare the two implementations over a grid of rational inputs, not just the displayed example. We can therefore say the abstraction exposes control structure; we cannot say that it enables a function ordinary software could not compute.

1.5 Development: constructing the working system

Expression selects or produces activity for a context. **Development** asks a related but distinct question: how is an operational system constructed from a more compact persistent description?

Biological development mediates the relation between genotype and phenotype; it is not a static one-to-one lookup from a DNA string to a finished organism.[3] Environmental influences can affect gene expression during development.[4] An engineering analogy should therefore make the construction process explicit:

$$P = \text{Dev}(G, E).$$

Here G is a software description, E a construction environment, and P a program. Unlike the organism, this particular P is a digital artifact with defined execution semantics. In a richer

model, development could depend on time, feedback, and stochastic events; the equation alone supplies none of them.

Our minimal generator takes a positive integer k and constructs a plan containing k copies of prefix. For $k = 2$, the input $(3, -1, 2)$ becomes $(3, 2, 4)$ after the first pass and $(3, 5, 9)$ after the second.



Generating a program is already possible in ordinary software.

Figure 1.5: A compact description constructs an ordered program before that program processes data. Repeating prefix twice yields the displayed exact trace. This original program generator is an analogy for description-to-structure construction, not a model of embryogenesis.

The construction already exists in ordinary programming. Its research value would have to come from how descriptions compose, how variation changes their products, or what constraints the development process enforces. A small description might generate a large repeated structure, but executing that structure still costs time and memory. Compression of description is not free execution.

This example also separates validity from fitness. Every positive k produces a well-typed plan, but increasing k need not improve a task objective. A type checker can reject malformed structure; it cannot replace an experiment measuring whether the structure solves the intended problem.

1.6 Execution, learning, and structural change

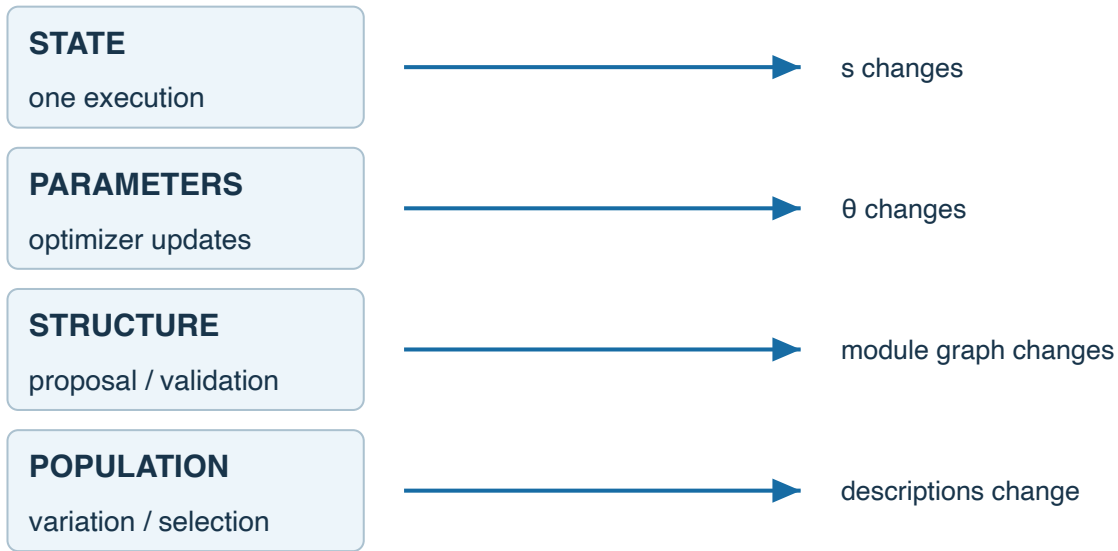
The same system can contain several update processes. A runtime state can change on every input while parameters remain fixed. An optimizer can change parameters while topology remains fixed. A structural procedure can add a module, alter a connection, or replace a description.

A candidate formal separation is

$$\begin{aligned}
 (s_{t+1}, y_t) &= F_{G, \theta}(s_t, x_t), \\
 \theta_{j+1} &= U(\theta_j, G, \mathcal{D}_j), \\
 G_{k+1} &= A(G_k, \mathcal{E}_k).
 \end{aligned}$$

The indices deliberately differ. t counts execution steps, j parameter updates, and k structural proposals. $\mathcal{D}_j$ is learning data and $\mathcal{E}_k$ the evidence used to accept, reject, or revise a proposal.

These equations describe possible interfaces; the chapter’s exact program implements neither U nor A .



Different update rules; no universal ordering of physical durations.

Figure 1.6: Four update roles: state, parameters, structure, and a population of inherited descriptions. Arrow meanings are update relations. No numerical biological or computational duration is asserted, and the rows do not impose a universal speed ordering.

One might run many inference steps between optimizer updates and many training runs between structural changes. That schedule is a design choice, not a theorem. Online learning can update parameters on every sample; program generation can happen in a millisecond; structural search can run offline for days.

An important engineering issue appears as soon as G changes: what happens to s and θ ? If the new structure has different state dimensions, blindly restoring an old checkpoint is invalid. A structural transition needs a migration map, a reset policy, or rejection. For example, replacing a state vector in $\mathbb{R}^d$ with one in $\mathbb{R}^{d'}$ requires specifying a map $\mathbb{R}^d \rightarrow \mathbb{R}^{d'}$ or explicitly discarding the old state.

This is where the genome analogy may become productive. An architecture that makes variation, construction, and migration explicit could be easier to reason about than ad hoc topology changes. Whether that organization improves learning or robustness is a separate empirical question.

1.7 Before calling it genomic, check computer science

The novelty burden is substantial because the neighboring fields are rich. Dynamic dispatch selects behavior from persistent code. Plugins extend a catalog. Query planning constructs an execution plan. Our exact example is already expressible with these ordinary ideas.

Mixture-of-experts systems route examples through selected subnetworks; Shazeer and colleagues’ sparse gate is a concrete established example.[5] Attention computes context-dependent weighted combinations of value representations from query–key scores.[6] Neither activity requires persistent architecture modification.

Neural architecture search can generate and evaluate model descriptions, as in Zoph and Le’s reinforcement-learning construction.[7] Program synthesis seeks a program meeting a specification;[8] genetic programming uses evolutionary search over programs.[9] These are direct neighbors of computational development and structural adaptation, not optional footnotes.

State-space models provide another established way to organize sequence computation. Mamba makes selected state-space parameters depend on the input so the recurrence can retain or discard information conditionally.[10] Agent systems also already compose reasoning with actions that obtain new information; ReAct provides one concrete example.[11]

Dispatch / plugins	select stored routines
MoE / conditional networks	route work among experts
Attention / retrieval	query accessible representations
Recurrence / SSMS	update carried state
Synthesis / architecture search	construct or search programs
Agent / workflow systems	compose tools and actions

A new label must earn a new semantic or empirical contribution.

Figure 1.7: Established alternatives organized by the role they already serve. The same proposed genomic mechanism may need comparison with several rows. This is a conceptual comparison map, not a benchmark or an exhaustive taxonomy of computer science.

A serious proposal should now answer a specific question. Does it offer a new semantics for compositional change? A useful inductive bias? A resource advantage on a defined task family? Better isolation of modifications? Or only a helpful vocabulary? Any of these may be worth studying, but they are different contributions.

For our two-context example, the direct baseline is functionally equivalent. If the plan interpreter is slower, a claim of inference efficiency fails for that implementation. It may still improve inspectability because the plan and trace are explicit artifacts. Such a result would support a tooling argument, not a new learning architecture.

A future comparison should match training data, task distribution, quality criterion, and relevant resource budgets. An ablation removing the proposed regulation mechanism should preserve other capacity as far as possible. Otherwise a larger catalog or extra training may explain the gain. This is the minimum experiment needed to turn a naming proposal into an engineering question.

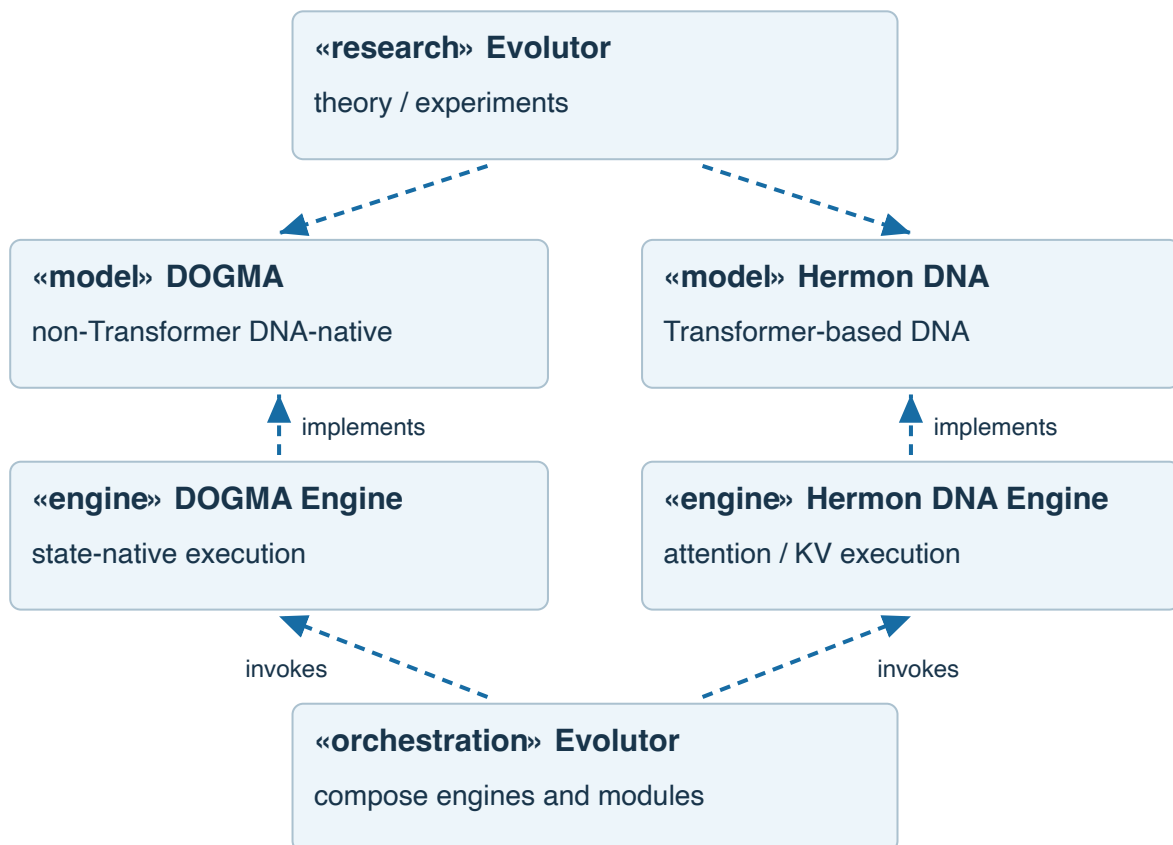
1.8 Evolutor, DOGMA, and Hermon DNA

This book uses three project levels with fixed meanings.

Evolutor is the broader genomic-computation theory and research framework, including the planned compiler/runtime and orchestration layer above model families. It may eventually coordinate models, retrieval, symbolic modules, and tools. That is a design hypothesis, not a deployed capability established here.

DOGMA names the non-Transformer DNA-native architecture research line and its corresponding non-Transformer inference engine, DOGMA Engine. The opening hypothesis emphasizes carried state, regulation, and expression.

Hermon DNA names the Transformer-based DNA architecture research line and its corresponding Transformer inference engine, Hermon DNA Engine. Its opening hypothesis retains attention and key/value context representations.



Dashed arrows: client depends on target; proposed roles only.

Figure 1.8: UML component-style research taxonomy. Evolutor contains the research and orchestration roles above the two families. DOGMA and Hermon DNA have separate model and engine identities. Dashed arrows express design relationships, not deployment or verified superiority.

In this project vocabulary, DNA-native and DNA-aware are research intentions whose operational content must be supplied by later representations, objectives, and mechanisms. They do not imply that either model runs in a test tube. Physical DNA computing, genomic sequence modeling, and biologically inspired digital organization must remain distinguishable.

An architecture specifies computations and state. An engine implements those computations under a resource and numerical contract. The engine may optimize layouts or kernels without

changing the intended model. Conversely, a changed state transition is an architectural change, even if introduced inside an implementation.

This distinction prevents a roadmap from collapsing into a list of products. We first need formal semantics and a reference model, then tests of equivalence, then evidence from experiments, then justified optimization. Naming an engine before those steps does not make it available.

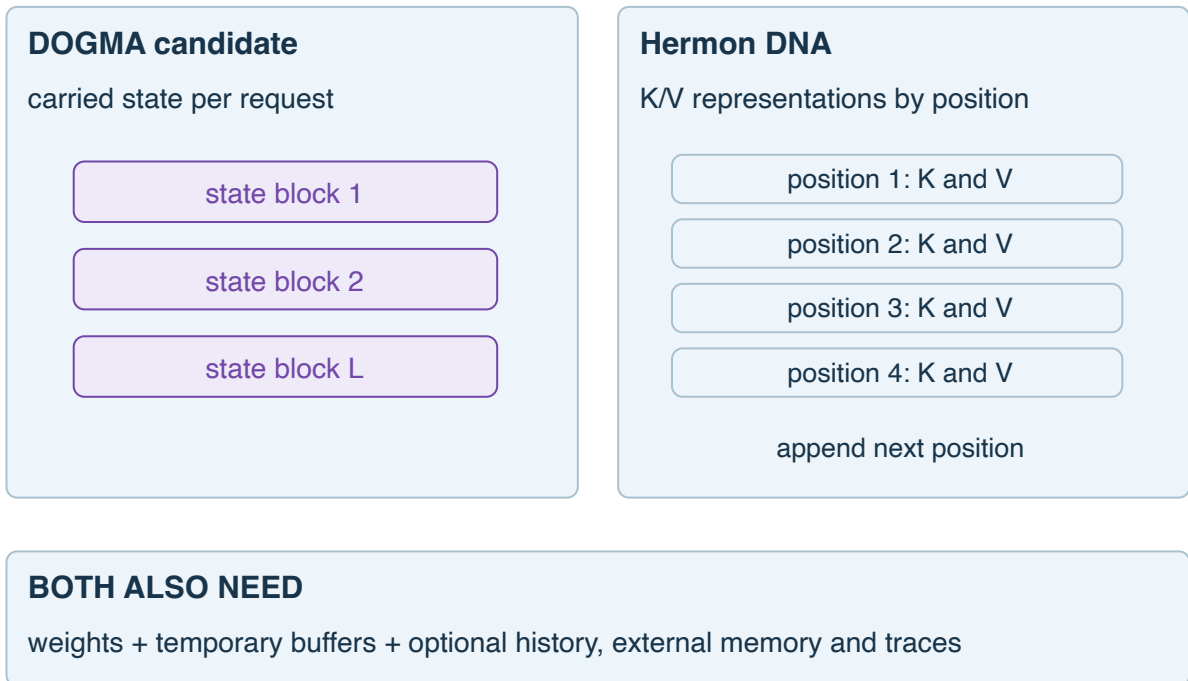
1.9 Two memory strategies, one honest budget

A recurrent candidate can summarize prior input in carried state:

$$(s_{t+1}, y_t) = F_\theta(s_t, x_t).$$

The state is sufficient only for the behavior the model can actually support. With fixed finite precision, a bounded state cannot preserve arbitrarily many distinguishable histories without collisions.

A causal attention model instead can retain key/value representations indexed by past position. Let C_{t-1} contain the cached pairs for positions $1, \dots, t-1$. Processing x_t appends (k_t, v_t) , forms C_t , and computes an output using the permitted prefix. The indexing matters: using a representation from a future token would violate the causal contract.



Compression and addressability are strategies, not a universal ranking.

Figure 1.9: Two candidate per-request memory organizations. Carried state transforms a bounded representation; a conventional full-context KV cache retains position-indexed representations. Both also require weights and workspace and may add external memory, traces, or history. The figure does not claim equal capability.

Attention's basic operation is

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \mathcal{M}\right) V,$$

where $Q \in \mathbb{R}^{q \times d_k}$, $K \in \mathbb{R}^{T \times d_k}$, and $V \in \mathbb{R}^{T \times d_v}$. The mask $\mathcal{M}$ excludes forbidden positions, and softmax acts across the T key positions for each query. This is an established mechanism, not a proposed DOGMA operation.[6]

Now derive a deliberately hypothetical storage comparison. Suppose batch size is B , layer count L , retained positions T , key/value head count H_{kv} , head width d_h , and storage per scalar b bytes. A conventional uncompressed KV layout needs

$$M_{KV} = 2BLTH_{kv}d_hb.$$

The factor two counts keys and values. A design storing d_s carried-state scalars per layer instead needs

$$M_s = BLd_sb.$$

These formulas count only the declared persistent per-request arrays. They omit weights, temporary activations, allocator overhead, retrieval stores, and any additional recurrent history.

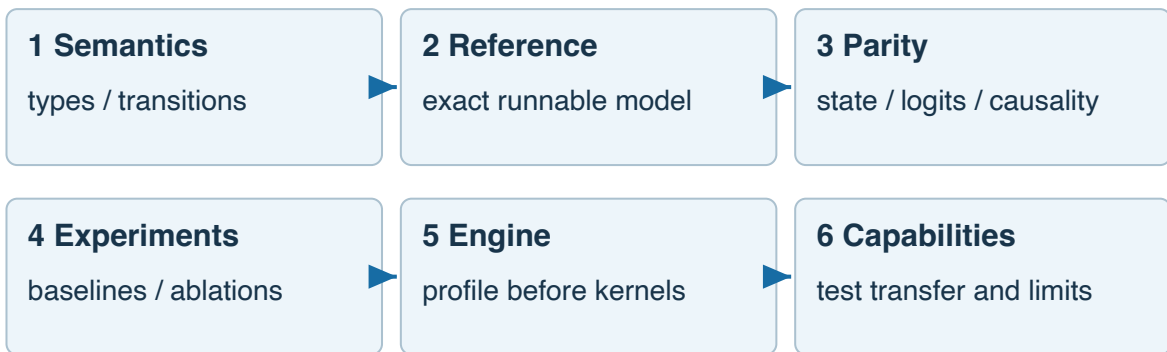
Take $B = 1$, $L = 24$, $T = 4096$, $H_{kv} = 8$, $d_h = 64$, $d_s = 2048$, and $b = 2$. The exact helper returns 201,326,592 bytes for KV, or 192 MiB, and 98,304 bytes for state, or 96 KiB. Here MiB and KiB mean powers of 1024. The array-size ratio is 2048.

That ratio is *not* a measured speedup and does not compare equal-quality models. A smaller state may discard distinctions a task needs. A cache may be windowed, quantized, shared, compressed, or supplemented by other memory. DOGMA is not proved superior by substituting a chosen state width into an accounting formula.

The productive research question is conditional: for a specified task and error tolerance, how much information must remain accessible, how is it updated or addressed, and what does that cost? Both families exist to explore different answers rather than to predeclare a universal winner.

1.10 A research program with concrete stopping points

The worked program has already taught several useful distinctions. A catalog differs from a plan. A plan differs from its execution trace. Generating a larger plan differs from learning it. Changing runtime values differs from changing persistent structure. These are ordinary, precise concepts on which a more ambitious proposal can be built.



A failure sends the proposal back for revision; no gate is proof of AGI.

Figure 1.10: A research sequence from semantics to evaluated capabilities. Reference behavior precedes engine optimization; parity checks precede performance claims. Failure at a gate sends a proposal back for revision. These are planned investigations, not completed model or AGI results.

The next steps should preserve that precision. Define a state transition before naming a stateful model. Define what regulation may inspect before claiming causal inference. Define how development preserves types before allowing generated structure to execute. Define migration and evaluation before treating a structural update as improvement.

Evolutor ultimately studies mechanisms relevant to memory, modularity, adaptation, continual learning, structural learning, tool composition, and generalization. Those motivations connect to broad questions about intelligence, but Chapter 1 establishes no AGI capability. A useful negative result—for example, exact equivalence to a simpler routing baseline—can clarify which mechanisms deserve further work.

Book I supplies the molecular-computation foundation. Its new Chapter 1 teaches a particular graph-search construction and its limits; it does not supply all later biology, thermodynamics, and circuit prerequisites. The shared dependency plan remains explicit about future teaching. Readers may bring equivalent background, but a roadmap reference is not proof that a prerequisite chapter already exists.

Chapter 2 develops learning objectives, data, and evaluation: sequence likelihood, leakage-resistant splits, and held-out generalization. These foundations make later genomic proposals testable. It does not jump directly to an optimized engine. This chapter’s standard is the model for the rest of the book: use abstraction freely, make its semantics concrete, and explain the difficult part directly.

1.11 Exercises

1. Classify changing a context, changing a weight, adding a module, and recording a trace by the role each changes.
2. Compute both plans on $(-2, 4, 1)$ and verify their stated invariants.
3. Prove that the final entry of prefix after center is zero for every nonempty rational input. Does the proof require all other prefix entries to be nonnegative?
4. Find an input on which clip and prefix do not commute. Explain why an unordered active set is insufficient.
5. Describe a test that distinguishes a changed plan from a changed persistent catalog.
6. Run the development rule for $k = 3$ on $(3, -1, 2)$. Which properties are guaranteed by type compatibility, and which are not?
7. Propose a structural migration from $\mathbb{R}^d$ to $\mathbb{R}^{d+1}$. State what it preserves and what performance claim it does not justify.
8. Double retained context length T in the hypothetical memory example. Then double batch size. Which arrays change, and why is this not a throughput prediction?
9. Compare the exact routed example with its direct baseline. Formulate one legitimate tooling claim and one unsupported architecture claim.
10. Design a future ablation of a regulation mechanism, specifying a task, a matched baseline, an intervention, and a possible negative result.

1.12 Worked solutions and discussion

1. Context changes the selection input; a weight update changes parameters; adding a module changes stored structure; recording a trace changes an observation artifact. None alone establishes evolutionary adaptation.
2. Bounded: $(-2, 4, 1) \rightarrow (0, 4, 1) \rightarrow (0, 4, 5)$. Balanced: mean 1, then $(-3, 3, 0) \rightarrow (-3, 0, 0)$. The first is nondecreasing and nonnegative; the second ends at zero.
3. The final prefix is $\sum_i (x_i - \bar{x}) = \sum_i x_i - m\bar{x} = 0$. No sign constraint on partial sums follows; $(-3, 0, 0)$ supplies a counterexample to nonnegativity.
4. For $(3, -1, 2)$, prefix after clip is $(3, 3, 5)$, while clip after prefix is $(3, 2, 4)$. Both plans use the same two names, so order must be represented.
5. Snapshot the immutable catalog identity and contents, vary only the context, and compare selected plans and traces. A structural-change test instead versions the catalog and replays the same input and context.
6. The third prefix pass turns $(3, 5, 9)$ into $(3, 8, 17)$. Composition preserves nonempty rational sequence type and length. It does not guarantee bounded values, useful predictions, or improved performance.
7. $m(s) = (s, 0)$ preserves the first d coordinates. It does not preserve the full model's behavior unless the new transition and readout are also constrained appropriately.
8. With doubled T , KV becomes 384 MiB while the declared state remains 96 KiB. Doubling B then gives 768 MiB and 192 KiB. Runtime depends on computation, bandwidth, batching, and implementation, none measured by these formulas.
9. The plan and trace expose selected operations for inspection. That supports an inspectability claim for this implementation. It does not show a function inaccessible to ordinary software or a learning advantage.
10. One possible task is context-dependent sequence processing under a fixed total parameter and data budget. Replace learned regulation with a matched fixed routing rule while preserving modules; measure held-out error and resource costs. Equal performance would fail to support a benefit from the learned regulator on that task. It would not establish universal uselessness.

Chapter 1 glossary

Stored structure The persistent description available for future activity. See Section 1.2.

Expressed computation The ordered activity selected or constructed for a context. See Section 1.2.

Adapted structure A persistent description after a specified update process. See Section 1.2.

Gene expression Production of functional RNA or protein from genomic information. See Section 1.3.

Regulation Biological control of activity; in the proposed software interface, a context-dependent control decision. See Section 1.3.

Plan An ordered list of operations with defined composition semantics. See Section 1.4.

Development Biological construction processes; by explicit analogy, a software description-to-program map. See Section 1.5.

State migration A declared way to move or reset state when structure changes. See Section 1.6.

Novelty burden The need to identify a contribution beyond established alternatives. See Section 1.7.

DOGMA The non-Transformer DNA-native model and corresponding engine research line. See Section 1.8.

Hermon DNA The Transformer-based DNA model and corresponding engine research line. See Section 1.8.

Evolutor The broader research, compiler/runtime and orchestration framework above both families. See Section 1.8.

KV cache Stored key/value representations for retained context positions. See Section 1.9.

Parity Agreement with a specified reference behavior under declared tolerances. See Section 1.10.

Bibliography

- [1] National Human Genome Research Institute. Gene Expression. [Source link](#).
- [2] Bruce Alberts et al. An Overview of Gene Control. Molecular Biology of the Cell, 4th edition (2002). [Source link](#).
- [3] Scott F. Gilbert. Principles of Development: Genes and Development. Developmental Biology, 6th edition (2000). [Source link](#).
- [4] Scott F. Gilbert. Mechanisms for the environmental regulation of gene expression. Birth Defects Research C 72 (2004), 291–299. [Source link](#).
- [5] Noam Shazeer et al. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer (2017). [Source link](#).
- [6] Ashish Vaswani et al. Attention Is All You Need (2017). [Source link](#).
- [7] Barret Zoph and Quoc V. Le. Neural Architecture Search with Reinforcement Learning (2017). [Source link](#).
- [8] Sumit Gulwani, Alex Polozov, and Rishabh Singh. Program Synthesis (2017). [Source link](#).
- [9] John R. Koza. Research overview: genetic programming. [Source link](#).
- [10] Albert Gu and Tri Dao. Mamba: Linear-Time Sequence Modeling with Selective State Spaces (2023; revised 2024). [Source link](#).
- [11] Shunyu Yao et al. ReAct: Synergizing Reasoning and Acting in Language Models (2023). [Source link](#).

Index

Adapted structure, 2

Development, 6

DOGMA, 10

Evolutor, 10

Expressed computation, 2

Gene expression, 3

Hermon DNA, 10

Stored structure, 2